As with stubs, Mocha provides a way to create whole mocks that exist just as mocks, as well as a way to create partial mocks that add expectations to an existing object. The method for whole-mock creation is mock:

```
test "a sample mock" do
  mocky = mock(name: "Paul", weight: 100)
  assert_equal("Paul", mocky.name)
end
```

The method for adding a mock expectation to a message on an existing object is expects. (I know, you'd think they'd have used mocks. But they didn't.)

minitest/01/test/models/project_with_stubs_test.rb

```
test "let's mock an object" do
  mock_project = Project.new(name: "Project Greenlight")
  mock_project.expects(:name).returns("Fred")
  assert_equal("Fred", mock_project.name)
end
```

Mocha has its own set of methods that can be chained to the mock declaration to add specifics:

```
proj = Project.new
proj.expects(:name).once
proj.expects(:name).twice
proj.expects(:name).at_least_once
proj.expects(:name).at_most_once
proj.expects(:name).at_least(3)
proj.expects(:name).at_most(3)
proj.expects(:name).times(5)
proj.expects(:name).times(4..6)
proj.expects(:name).never
```

If you have multiple return values specified, the stubbed method returns them one at a time:

minitest/01/test/models/project_with_stubs_test.rb

```
test "stub with multiple returns" do
  stubby = Project.new
  stubby.stubs(:user_count).returns(1, 2)
  assert_equal(1, stubby.user_count)
  assert_equal(2, stubby.user_count)
  assert_equal(2, stubby.user_count)
end
```

As with RSpec, the last value is repeated over and over again.

You can get the same effect with a little more syntactic sugar by using the then method. You can chain together as many of these as you want:

```
stubby.stubs(:user_count).returns(1).then.returns(2)
```